Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

01

MODULE 01

Understanding Git & GitHub

What version control is, why every developer uses it, and how Git, GitHub and VS Code fit together.

MODULE 01 · LESSON 1.1

What is version control?

ما هو التحكم بالإصدارات؟

Learning objectives

- Explain what version control is in your own words.
- Describe three problems caused by copying project folders by hand.
- Explain how Git records changes as a chain of commits.
- Tell the difference between local and distributed version control.

Prerequisites

None. You only need a computer and a project, even a simple text file, that you have edited more than once.

Introduction

Imagine you are writing an important report. On Monday you save `report.docx`. On Tuesday you change it and save `report_final.docx`. By Friday you have `report_final2.docx` and `report_REAL_final.docx`, and you are not sure which one is right. Version control solves exactly this problem, and it does it for every file in a project at once.

Explanation

Version control is a system that records every change made to a set of files, who made it, when, and why. You can go back to any earlier moment, compare two moments, and work with other people without overwriting each other.

Why developers need it. Code changes constantly. A new feature can break something that worked yesterday. With version control you can see exactly what changed and undo it safely. Teams also need it: five people can edit the same project and combine their work.

The problem with copying folders. Copies take space, names become confusing, nothing explains *why* a copy exists, and two people copying the same folder can never merge their changes automatically.

How Git works. Git does not keep random copies. When you tell it to, it saves a **commit**: a snapshot of your project plus a message and an author. Each commit points to the one before it, so your history is a chain you can walk backwards.

Local and distributed. Older systems kept the history on one central server. Git is **distributed**: every developer has the full history on their own computer. You can work offline, and if a server is lost, any copy can restore it.

Real-world analogy

> Think of a video game with save slots. Each save is a moment you can return to. A commit is a save point for your project, and the message is the label you write on it, such as "Before the boss fight".

Diagram

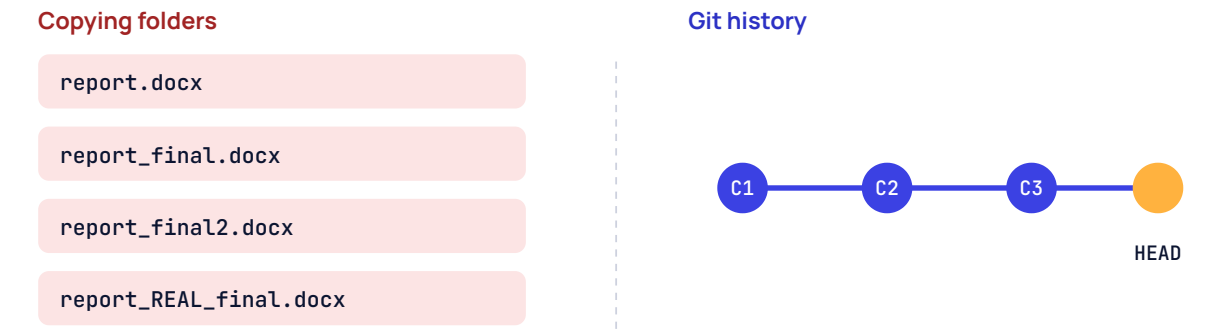


Figure 1 · Copying folders vs. a Git history

Commands

There are no commands in this lesson. In Lesson 2.1 you will install Git and run your first one, `git --version`.

Common mistakes and solutions

"Git is a backup tool."	Git records meaningful changes you choose to save. Keep real backups too.
"I must be online to use Git."	Git works fully offline. You only need internet to share with GitHub.
"Version control is only for big teams."	It helps a single person from the first day of a project.

Practical exercise

> Find a project or document you have edited many times. Write down three situations where you wished you could go back or compare versions. For each one, note which Git idea (history, commit, or sharing) would help.

Expected result

A short list of three real situations, each matched to an idea from this lesson. There is no right or wrong list. It prepares you for Lesson 1.2.

Knowledge check

1. What does a commit store?

- ○ A copy of the internet
- ○ A snapshot of the project, a message, and an author
- ○ Only the names of changed files

2. Why is Git called distributed?

- ○ Every developer has the full history locally
- ○ It only runs on one server
- ○ It distributes free files

3. Which is a problem of copying folders by hand?

- ○ Windows deletes the copies
- ○ Unclear names and no explanation of changes
- ○ Files become read-only

Answers

- A snapshot of the project, a message, and an author — A commit is a snapshot with a message and author, linked to the commit before it.
- Every developer has the full history locally — Each copy contains the whole history, so work does not depend on one central server.

- Unclear names and no explanation of changes — Names like final2 do not say what changed or why, and copies cannot be merged automatically.

Summary

Version control records the history of a project. Git saves snapshots called commits, keeps the full history on every computer, and lets you return to any point or combine work from several people.

Key vocabulary

Version control	A system that records changes to files over time
Commit	A saved snapshot with a message and author
History	The chain of commits in a project
Distributed	Every copy holds the full history

Next lesson

> **Lesson 1.2 — Git vs GitHub vs VS Code.** You now know what Git does. Next you will meet the three tools you will use and see how they work together.

MODULE 01 · LESSON 1.2

Git vs GitHub vs VS Code

Git مقابل GitHub مقابل VS Code

Learning objectives

- Define Git, GitHub and VS Code in one sentence each.
- Explain local and remote repositories.
- Tell public repositories from private ones.

Prerequisites

Lesson 1.1.

Introduction

Beginners often use “Git” and “GitHub” as if they were one thing. They are three different tools that work together, and knowing the difference removes most early confusion.

Explanation

Git is a free program on your computer. It records the history of your files. It works without internet.

GitHub is a website that stores Git projects online. It adds sharing and teamwork: backups, issues, pull requests and a profile page.

VS Code (Visual Studio Code) is a code editor. It has Git built in, so you can stage, commit and push with buttons instead of typing.

A **repository** (repo) is a project folder plus its Git history. Your copy is the **local repository**. The copy on GitHub is the **remote repository**. A **commit** is one saved snapshot, and a **branch** is a line of commits. You will study branches in Module 7.

Public repositories can be seen by anyone. **Private** repositories are visible only to you and people you invite. Never put passwords in either kind.

Real-world analogy

> Writing a book: VS Code is your writing desk, Git is the editor who keeps every draft, and GitHub is the shared cloud folder your team reads.

Diagram

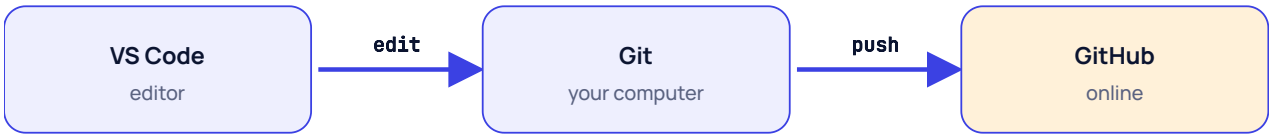


Figure 1 · How the three tools fit together

Commands

No commands yet. In Lesson 2.1 you will install the tools and run `git --version`.

Common mistakes and solutions

Thinking GitHub is required to use Git.	Git works alone. GitHub is optional, but very useful for sharing.
Thinking VS Code and Git are the same.	VS Code is an editor. It only gives you buttons for Git.
Making a repo public by accident.	Check the visibility setting before you create or publish a repository.

Practical exercise

> Open github.com in your browser. Open any public repository. Find three things: the README text, the list of files, and the branch name (usually `main`).

Expected result

You can point to the README, the file list and the branch selector on the page. You did not need an account for this.

Knowledge check

1. Which tool stores your project online?

- ○ Windows Terminal
- ○ GitHub
- ○ VS Code

2. Where is the local repository?

- ○ On your computer
- ○ Only on GitHub
- ○ Inside the browser

3. Who can see a private repository?

- ○ Only GitHub staff
- ○ Only you and people you invite
- ○ Everyone

Answers

- GitHub — GitHub hosts remote repositories.
- On your computer — The local repository is the project history stored on your own machine.
- Only you and people you invite — Private means visible only to you and chosen collaborators.

Summary

Git tracks changes, GitHub hosts repositories online, and VS Code is the editor with Git buttons. Your computer holds the local repository, GitHub holds the remote one.

Key vocabulary

Repository	A project folder with its Git history
Local / Remote	On your computer / on GitHub
Public / Private	Visible to everyone / only to invited people
Branch	A line of commits

Next lesson

> **Lesson 1.3 — The Git workflow.** Next you will follow a file from your editor to GitHub, stage by stage.

MODULE 01 · LESSON 1.3

Understanding the Git workflow

فهم سير عمل Git

Learning objectives

- Name the four places a change can live.
- Recognize untracked, modified, staged and committed files.
- Explain add → commit → push.

Prerequisites

Lessons 1.1 and 1.2.

Introduction

Git does not save everything automatically. You choose what to save and when. That choice happens in stages, and learning them makes every later command easy to understand.

Explanation

Working directory: your project folder, where you edit files.

Untracked files are new files Git has never seen. **Modified** files are tracked files you changed since the last commit.

Staging area: a waiting room. You place the changes you want in the next commit here. The command is `git add`.

Local repository: the history on your computer. `git commit` saves the staged changes as a new commit.

Remote repository: the copy on GitHub. `git push` sends your new commits there.

The **commit history** is the list of all commits. **Remember:** staging is not committing, and committing is not pushing. Each is a separate step.

Real-world analogy

> Packing a parcel: you edit in your room (working directory), put the items you chose into the box (staging), seal and label it (commit), then send it by post (push).

Diagram

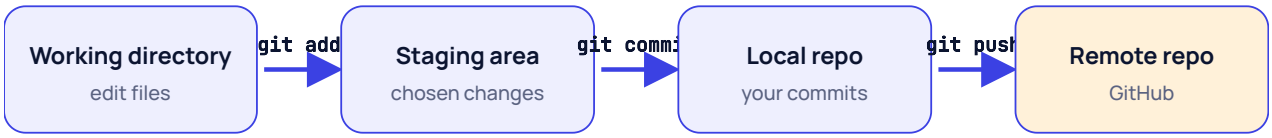


Figure 1 · The add → commit → push workflow

Commands

You will learn these commands in Modules 4 and 5. For now, just recognize them: `git add`, `git commit -m "message"`, `git push`.

Common mistakes and solutions

Thinking add saves the work.	It only prepares changes. Nothing is saved until you commit.
Thinking commit uploads to GitHub.	Commit saves on your computer only. Push uploads.
Expecting new files to be tracked automatically.	New files are untracked until you add them.

Practical exercise

> Imagine you edit `index.html`, then run `git add index.html`, then `git commit -m "Add homepage"`. Write which stage the change is in after each of the three actions.

Expected result

After editing: modified, in the working directory. After add: staged. After commit: saved in the local repository, not yet on GitHub.

Knowledge check

1. Which command sends commits to GitHub?

- ○ git commit
- ○ git push
- ○ git add

2. A brand-new file Git has never seen is…

- ○ Untracked
- ○ Committed
- ○ Pushed

3. Does git add create a commit?

- ○ Only on Fridays
- ○ No, it only stages changes
- ○ Yes

Answers

- git push — push uploads commits to the remote repository.
- Untracked — New files are untracked until you add them.
- No, it only stages changes — Staging prepares changes. Commit creates the snapshot.

Summary

A change travels from the working directory to the staging area (add), to the local repository (commit), and to the remote repository (push).

Key vocabulary

Working directory	Your project folder
Staging area	Where changes wait for the next commit
Untracked / Modified	New file / changed tracked file
Push	Send commits to the remote

Next lesson

Module 2 — Installing and configuring Git. You know the theory. Now you will install the tools and run your first command.

MODULE 01

Practical lab 1 · Identify every stage

Read the scenario and fill in the table. Scenario: you create `style.css`, edit `index.html`, then run `git add index.html`, `git commit -m "Add homepage"`, and `git push`.

Moment	Where is `index.html`'s change?	Where is `style.css`?
After editing	Modified (working directory)	Untracked
After git add	Staged	Untracked
After git commit	Local repository	Untracked
After git push	Remote repository (GitHub)	Untracked

Evaluation: 4 correct rows = pass. Note that `style.css` stays untracked because it was never added.

Module 1 quiz (pass mark: 70%)

1. What is Git?

- ○ A website
- ○ A code editor
- ○ A program that tracks file history

2. What is GitHub?

- ○ A file format
- ○ A website that hosts Git repositories
- ○ A terminal

3. What does staging do?

- ○ Prepares changes for the next commit
- ○ Uploads to GitHub
- ○ Deletes files

4. After git commit, where are your changes?

- ○ Nowhere
- ○ In the local repository
- ○ On GitHub

5. Can Git work without internet?

- ○ No
- ○ Only on Windows
- ○ Yes

Answers

- A program that tracks file history — Git is the version control program on your computer.
- A website that hosts Git repositories — GitHub stores remote repositories online.
- Prepares changes for the next commit — Staging selects the changes for the next commit.
- In the local repository — Commit saves on your computer; push is the step that uploads.
- Yes — Git is distributed; the history is local.